



Faculty of Engineering & Technology

Department of Electrical & Computer Engineering

COMP333

DataBase Systems

BD project

Prepared by:

Name: Basmala Abu hakema

Number: 1220184

Section: 5

Name: Maysam Habbash

Number: 1220075

Section: 5

Instructor: Dr. Mohammad HussienAli

18th Jan 2026

Contents

Table of figures:	III
Table of tables:	III
1. Introduction.....	1
2. Project Overview	2
2.1. About LTC	2
2.2. Project Goals.....	2
2.3. Client Information.....	2
3. Technology Stack.....	3
3.1. Backend Technology	3
3.2. Frontend Technology	3
3.3. Database Management System	4
4. Database Schema Design.....	5
4.1. Overview.....	5
4.2. Core Entities and Their Purposes.....	6
4.2.1. User Entity	7
4.2.2. Product and Category Entities.....	7
4.2.3. Order and Order Item Entities.....	7
4.2.4. Payment Entity.....	8
4.2.5. Address Entity.....	8
4.2.6. Warehouse and Inventory Management.....	8
4.2.7. Warranty Entity	8
4.2.8. Sale Entity	8
4.3. Key Relationships	9
Entity Relationships Summary.....	9
5. Queries	10
CREATE (Adding new data).....	10
READ (Retrieving and viewing data).....	10
UPDATE (Modifying data).....	10
DELETE (Removing data).....	10
6. Database Normalization.....	11

6.1.	First Normal Form (1NF).....	11
6.2.	Second Normal Form (2NF)	11
6.3.	Third Normal Form (3NF)	12
7.	Database Operations and Workflow.....	13
7.1.	User Registration and Authentication	13
7.2.	Product Browsing and Search	13
7.3.	Shopping Cart Management.....	13
7.4.	Order Processing.....	14
7.5.	Order Tracking and Delivery	14
7.6.	Reporting and Analytics.....	15
	Conclusion	16

Table of figures:

Figure 1: ERD	5
---------------------	---

Table of tables:

Table 1: Entities	6
-------------------------	---

1. Introduction

This report documents the design and implementation of the database system for LTC, an e-commerce platform specializing in networking equipment, computers, and phone accessories. The project aimed to create a robust, normalized database that could handle all aspects of online retail operations, from product management to order processing and inventory tracking.

Our team developed this system to support a full-featured online store where customers can browse products, place orders, track deliveries, and manage their accounts. The database serves as the backbone of the entire application, ensuring data integrity and efficient operations across all user interactions.

2. Project Overview

2.1. About LTC

LTC is an e-commerce website that sells a wide range of technology products including networking equipment like routers and switches, computer hardware and accessories, and mobile phone products. The platform needed to handle multiple product categories, manage user accounts, process payments, track orders through delivery, and maintain warranty information for purchased items.

2.2. Project Goals

We wanted to build a system that could keep track of everything happening in the store: who our customers are, what products we have in stock, which warehouses hold our inventory, how orders move from placement to delivery, and how we manage warranties. The goal was to make sure all this information was organized properly, easy to access, and always accurate.

2.3. Client Information

Client Name: Light for Technology & Computer (LTC)

Location: Ramallah, Palestine

Industry: Technology, Computers & Networking Equipment

Client Needs: Expand business to an online environment, automate product and order handling, secure user data, and improve accessibility and sales.

3. Technology Stack

3.1. Backend Technology

For the backend, we chose **Python** with the **Flask framework**. Python was the natural choice because it's straightforward to work with and has excellent libraries for database operations. Flask is a lightweight web framework that gives us the flexibility to build exactly what we need without unnecessary complexity.

We structured our project using the **MVC** (Model-View-Controller) architecture pattern. This helped us organize our code logically and keep different concerns separated. The **Model layer** contains our database models and all the logic for interacting with data. The **View layer** handles presenting information to users through our **HTML templates**. The **Controller layer**, implemented through Flask routes, manages the flow of data between models and views, handling user requests and coordinating responses.

Using MVC made our development process smoother because team members could work on different parts of the application without stepping on each other's toes. If we needed to change how data was displayed, we could modify the views without touching the database logic. If we needed to add new database functionality, we could update the models without breaking the user interface. This separation of concerns is especially valuable in web applications where the same data might need to be presented in different ways.

3.2. Frontend Technology

For the frontend, we used **HTML** and **CSS** to build the user interface. We kept things simple and functional, focusing on creating clear pages for browsing products, browsing categories, viewing the shopping cart, checking out, and managing user profiles. HTML gave us the structure for all our pages, while CSS handled the styling to make everything look professional and easy to use.

3.3. Database Management System

MySQL was our choice for the database management system. It's reliable, widely used, and perfect for handling the transactional nature of an e-commerce site. MySQL gave us **ACID** compliance, which means our transactions are atomic, consistent, isolated, and durable. This is crucial when processing orders and payments where data integrity is absolutely essential.

MySQL also provided us with good performance for the types of queries we needed to run, whether that was searching for products, looking up order history, or generating reports in dashboards for all user types on sales and inventory.

4. Database Schema Design

4.1. Overview

Our database schema was designed to capture all the essential entities and relationships needed to run an e-commerce business. We identified the core entities: users, products, orders, payments, addresses, warehouses, and warranties. Each entity became a table in our database, and we carefully designed how these tables relate to each other to reflect real-world business processes.

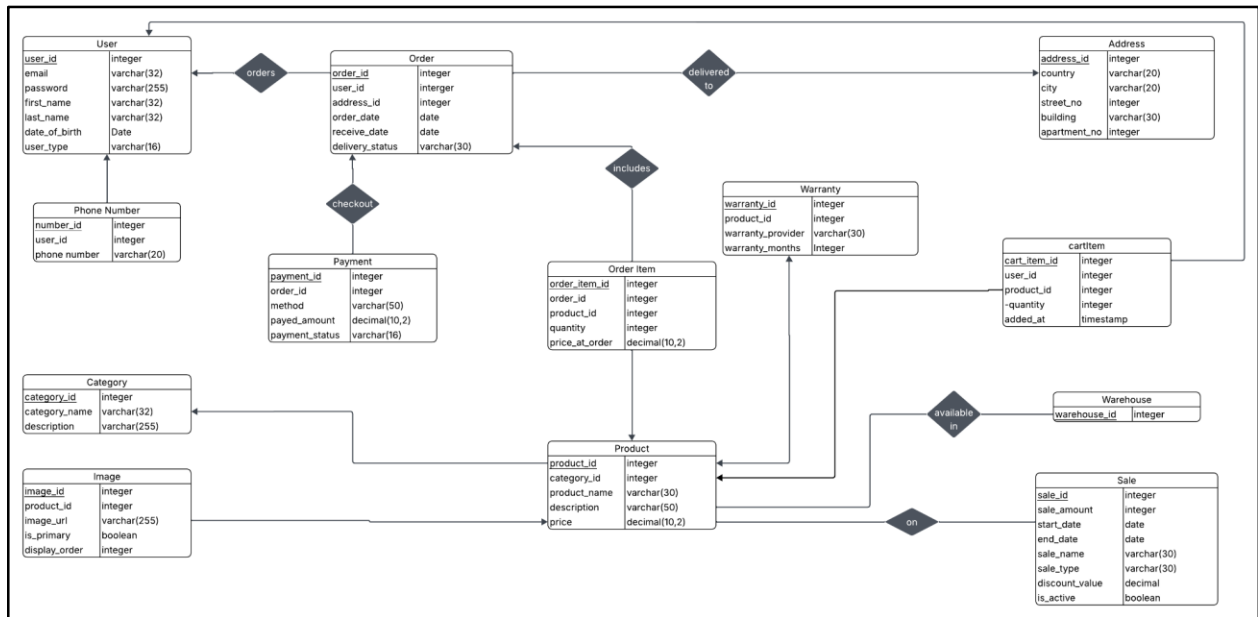


Figure 1: ERD

4.2. Core Entities and Their Purposes

Table 1: Entities

Entities	Attributes
warehouse	<u>Ware House ID</u>
Cart Item	<u>Cart Item ID</u> , user_id, product_id, quantity, added_at
Product	<u>Product ID</u> , category_id, product_name, price(), Description
Warranty	<u>Warranty ID</u> , product_id, warranty_provider, warranty_months
Order	<u>Order ID</u> , user_id, address_id, order date, receive date, delivery status
sale	<u>Sale ID</u> , sale_amount, start_date, end_date, sale_name, sale_type, discount_value, is_active
Payment	<u>Payment ID</u> , order_id, method, payed_amount(), payment_status
Order Item	<u>Order Item ID</u> , order_id, product_id, quantity, price_at_order()
User	<u>User ID</u> , email, password, first_name, last_name, date_of_birth, user_type
Phone Number	<u>Number ID</u> , user_id, phone_number
Category	<u>Category ID</u> , category_id, category_name, description
Address	<u>Address ID</u> , country, city, street_number, building, apartment_number
Image	<u>Image ID</u> , product_id, image_url, is_primary, display_order

4.2.1. User Entity

The **User** table stores information about everyone who creates an account on our platform. It includes basic details like *email*, *password*, *first* and *last names*, *date of birth*, and *user type*. We need to know who our customers are so they can log in, place orders, and track their purchases. The **user id** serves as the primary key, uniquely identifying each person in our system. Other user types include an employee and an admin, an admin is able to create accounts for employees to manage orders and warehouses.

Each user can have multiple phone numbers, which is why we created a separate Phone Number table. This allows customers to save both mobile and home numbers if they want. The relationship between User and Phone Number is one-to-many, meaning one user can have many phone numbers, but each phone number belongs to only one user.

4.2.2. Product and Category Entities

The **Product** table is where we store all our inventory items. Each product has a unique identifier, *name*, *description*, and *price*. Products are organized into **categories** like networking equipment, computers, or phone accessories, which helps customers browse and find what they need. The Category table maintains these groupings with a name and description for each category.

We also created an **Image** table to handle product photos. Each product can have multiple images, and each image knows its display order so we can show the main photo first and additional angles afterward. This one-to-many relationship between Product and Image gives us flexibility in how we present products to customers.

4.2.3. Order and Order Item Entities

When a customer decides to buy something, we create an **Order** record. This captures who made the order, when they placed it, when they wanted to receive it, the delivery status, and the total price. Each order can contain multiple products with different quantities, so we use an **Order Item** table as a junction between Order and Product. This many-to-many relationship lets us track exactly what was purchased in each order, how many of each item, and what the price was at the time of purchase.

4.2.4. Payment Entity

The **Payment** table records how customers pay for their orders. We track the *payment method* (credit card, debit card, PayPal, etc.), the *amount paid*, and whether the payment was successful. This is connected to the Order table through a checkout relationship.

4.2.5. Address Entity

The **Address** table stores delivery locations with complete details including *country*, *city*, *street number*, *building*, and *apartment number*. Orders are delivered to addresses, creating a many-to-one relationship where many orders can be delivered to the same address, but each order has one delivery destination.

4.2.6. Warehouse and Inventory Management

We use **warehouses** to store our products before they're sold. The Warehouse table is simple, containing just a unique identifier for each location. Products can be available in multiple warehouses, and each warehouse can stock many different products. This many-to-many relationship is implemented through the **WarehouseProducts** table, which doesn't just link products to warehouses but also tracks inventory movements.

4.2.7. Warranty Entity

The **Warranty** table manages warranty information for products. It stores the *warranty provider*, how many months the warranty lasts, and which order items it covers. This one-to-one relationship with Product means that each product can have warranty coverage, which is important for electronics and technology products that typically come with manufacturer warranties.

4.2.8. Sale Entity

The Sale table tracks promotional discounts on products. Each sale record includes the discount amount, start date, and end date. Products can be on sale at different times.

4.3. Key Relationships

The relationships in our schema reflect how the business actually operates. Users place orders, creating a one-to-many relationship. Orders contain products through Order Items, forming a many-to-many relationship with additional attributes like quantity and price. Products belong to categories and can have multiple images, both one-to-many relationships.

Orders are delivered to addresses, payments are made for orders through checkout, and products can be available in multiple warehouses. Each of these relationships was carefully designed to avoid data redundancy while maintaining the ability to answer important business questions like 'What products did this customer order?'

Entity Relationships Summary

1. Users → Phone_Number (1:N) - One user can have multiple phone numbers
2. Users → Orders (1:N) - One user can have multiple orders
3. Users → CartItem (1:N) - One user can have multiple cart items
4. Category → Product (1:N) - One category can have multiple products
5. Product → Image (1:N) - One product can have multiple images
6. Product → Warranty (1:N) - One product can have multiple warranty records (latest used)
7. Product → WarehouseProduct (1:N) - One product can be in multiple warehouses
8. Product → OrderItem (1:N) - One product can appear in multiple order items
9. Product → CartItem (1:N) - One product can be in multiple users' carts
10. Warehouse → WarehouseProduct (1:N) - One warehouse can have multiple products
11. Orders → OrderItem (1:N) - One order can have multiple items
12. Orders → Payment (1:N) - One order can have multiple payments
13. Address → Orders (1:N) - One address can be used for multiple orders
14. Sale → ProductSale (1:N) - One sale can apply to multiple products
15. Product ↔ Warehouse Products can be in multiple warehouses
16. Product ↔ Sale Products can have multiple sales, sales can apply to multiple products

5. Queries

CREATE (Adding new data)

1. Add a new product to the system.
2. Add a new customer account.
3. Add a new order made by a customer.
4. Add new stock quantity for a product in a warehouse.

READ (Retrieving and viewing data)

5. Total number of warehouses.
6. Number of employees.
7. Products available in a specific warehouse.
8. Quantity of a specific product in a specific warehouse and across all warehouses.
9. Total number of orders made by a specific customer.
10. Total revenue from orders per month or year.
11. Best-selling products.
12. All products purchased by one customer last month.
13. Out-of-stock products in a specific warehouse.
14. Total number of users registered on the website.
15. Total number of laptops in stock in a specific warehouse.
16. Available products in all warehouses.

UPDATE (Modifying data)

18. Update the price of a product.
19. Update product stock after a new shipment.
20. Update a customer's contact information.
21. Update an order status such as “Pending” to “Shipped” or “Delivered”.

DELETE (Removing data)

22. Delete a discontinued product.
23. Delete a canceled order.
24. Delete a customer account upon request.

6. Database Normalization

Normalization was a critical part of our database design process. We applied normalization principles to eliminate data redundancy and ensure that our database structure was efficient and maintainable. We achieved Third Normal Form (3NF) across all our tables, which means our data is well-organized and updates can be made without causing inconsistencies.

6.1. First Normal Form (1NF)

To achieve First Normal Form, we ensured that every table has a primary key and that all attributes contain only atomic values. This means no repeating groups or arrays within columns.

For example, instead of storing multiple phone numbers in a single field like 'phone_numbers: 123-456, 789-012', we created a separate Phone Number table where each phone number is stored in its own row. Similarly, we separated product images into their own table rather than storing multiple image URLs in a single Product record.

Every table in our schema has a clear primary key: `user_id` for Users, `product_id` for Products, `order_id` for Orders, and so on. This uniquely identifies each record and forms the foundation for relationships between tables.

6.2. Second Normal Form (2NF)

Second Normal Form requires that all non-key attributes depend on the entire primary key, not just part of it. This mainly affects tables with composite keys.

In our Order Item table, which links orders to products, we could have had a composite key of (`order_id`, `product_id`). However, we added `order_item_id` as the primary key and made sure all other attributes like `quantity` and `price_at_order` depend on this entire key, not just the order or product alone. The quantity and price are specific to this particular order item, not just to the product or order by itself.

We also made sure that product information like name and description stayed in the Product table rather than being repeated in Order Items. Order Items only store what's specific to that purchase: the quantity bought and the price at the time of order.

6.3. Third Normal Form (3NF)

Third Normal Form means removing transitive dependencies, where non-key attributes depend on other non-key attributes rather than directly on the primary key.

A good example is how we handled categories. Initially, we might have considered storing `category_name` directly in the `Product` table. However, this would mean that the `category_name` depends on `category_id`, which itself depends on `product_id`. This is a transitive dependency. To eliminate it, we created a separate `Category` table where category details are stored once, and products simply reference the `category_id`.

Similarly, we separated addresses into their own table. Instead of storing delivery address details directly in the `Order` table, we created an `Address` table and have orders reference `address_id`. This means if multiple orders go to the same address, we store that address information only once.

7. Database Operations and Workflow

The database supports all the critical operations needed to run our e-commerce platform. Here's how the website uses the database in its day-to-day workflow.

7.1. User Registration and Authentication

When someone wants to create an account, our system collects their information and inserts a new record into the User table. We hash the password before storing it for security. During login, we query the User table with the provided email, verify the password hash, and create a session for the authenticated user. This session is maintained throughout their visit, allowing them to browse, add items to cart, and eventually check out.

7.2. Product Browsing and Search

When customers visit our store, they see products pulled directly from the database. We run queries that join the Product table with Category and Image tables to display products with their categories and photos. Customers can filter by category, which translates to a WHERE clause in our SQL query. When someone searches for a specific item, we use LIKE operators on product names and descriptions to find matches.

For each product page, we also query the Sale table to check if there's an active discount, showing the reduced price if applicable. We query the Warehouse Transactions table to calculate total available inventory across all warehouses, displaying whether items are in stock.

7.3. Shopping Cart Management

Shopping cart data is temporarily stored in the session on the backend and stored in a Cart table. When customers add products to their cart, we query the Product table to get current prices and verify availability. The cart maintains product IDs and quantities until the customer proceeds to checkout. This approach keeps the database clean since we don't need to persist cart data for users who don't complete their purchase.

7.4. Order Processing

When a customer checks out, several database operations happen in a transaction. First, we create or retrieve an Address record for the delivery location. Then we insert a new Order record linked to the user and address. For each item in the cart, we create an Order Item record linking the order to the product with the quantity and price at the time of purchase.

Next, we handle payment by inserting a Payment record and linking it to the order through the checkout relationship. We validate that payment was successful before updating the order status to confirm. Finally, we create Warehouse Transaction records to deduct the purchased quantities from inventory. All of this happens within a single database transaction, so if any step fails, everything rolls back and the order isn't created.

7.5. Order Tracking and Delivery

Customers can view their order history by querying the Order table filtered by their `user_id`. For each order, we join with Order Items and Products to show what was purchased. The `delivery_status` field in the Order table tracks progress from processing to shipped to delivered. When the status changes, we update this field and customers see the latest status on their order tracking page.

The `delivered_to` relationship between Order and Address lets us display the delivery address on the order details page, and the `order_date` and `receive_date` fields show when the order was placed and when it's expected to arrive.

7.6. Reporting and Analytics

The database supports various business intelligence queries. We can calculate total sales by joining Orders with Order Items and aggregating the total_price. We can identify best-selling products by counting order items grouped by product_id. Category performance is analyzed by joining Products with Categories and Order Items to see which product categories generate the most revenue.

We can also track customer lifetime value by summing all orders for each user, identify inventory turnover rates by analyzing warehouse transactions, and monitor payment success rates through the Payment table. These insights help us make better business decisions about what products to stock, which promotions work best, and where to focus our marketing efforts.

Conclusion

This database project successfully created a comprehensive data management system for LTC's e-commerce operations. By carefully designing our schema with proper normalization, we built a database that efficiently handles all aspects of running an online store, from managing products and inventory to processing orders and tracking deliveries.

The combination of Python and Flask for the backend provided a solid foundation for database operations, while MySQL ensured data integrity and performance. Our normalized schema eliminates redundancy and maintains consistency, making the system easier to maintain and extend as the business grows.

The relationships between entities accurately reflect real-world business processes, and the database operations support the complete workflow of an e-commerce platform. From user registration through product browsing, cart management, order processing, and delivery tracking, every aspect is properly supported by the underlying database structure.

This project demonstrates how proper database design principles, when applied to a real-world business problem, create a system that is both functional and scalable. The LTC e-commerce platform now has a robust data foundation that can support current operations and adapt to future business needs.